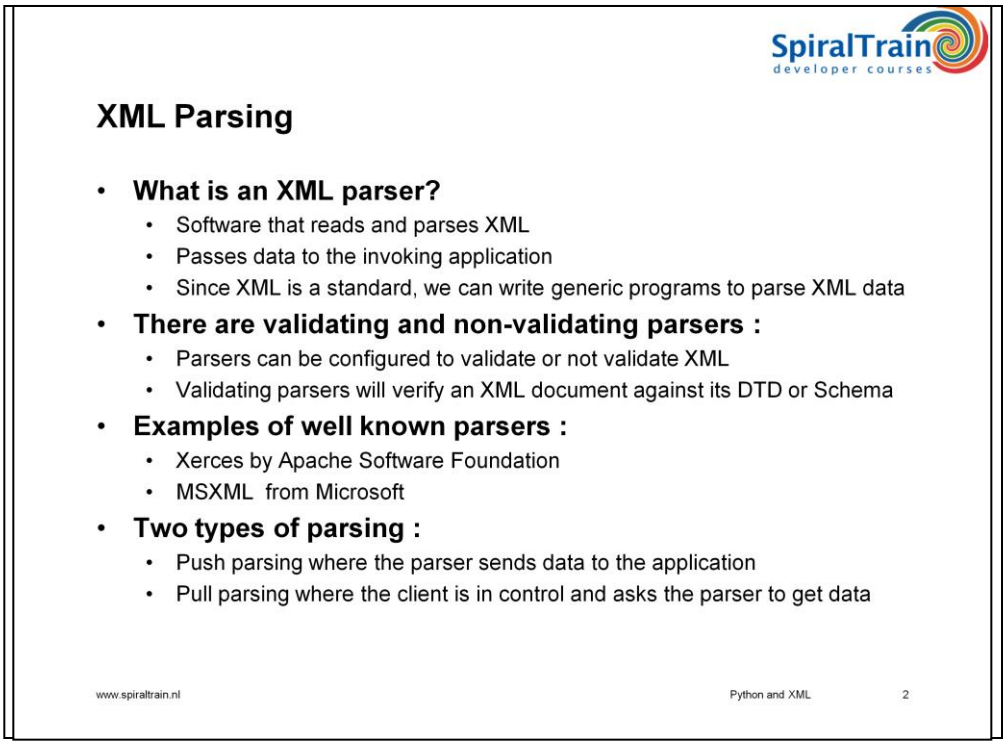
SpiralTrain
developer courses

Course Schedule

- **Python and XML**

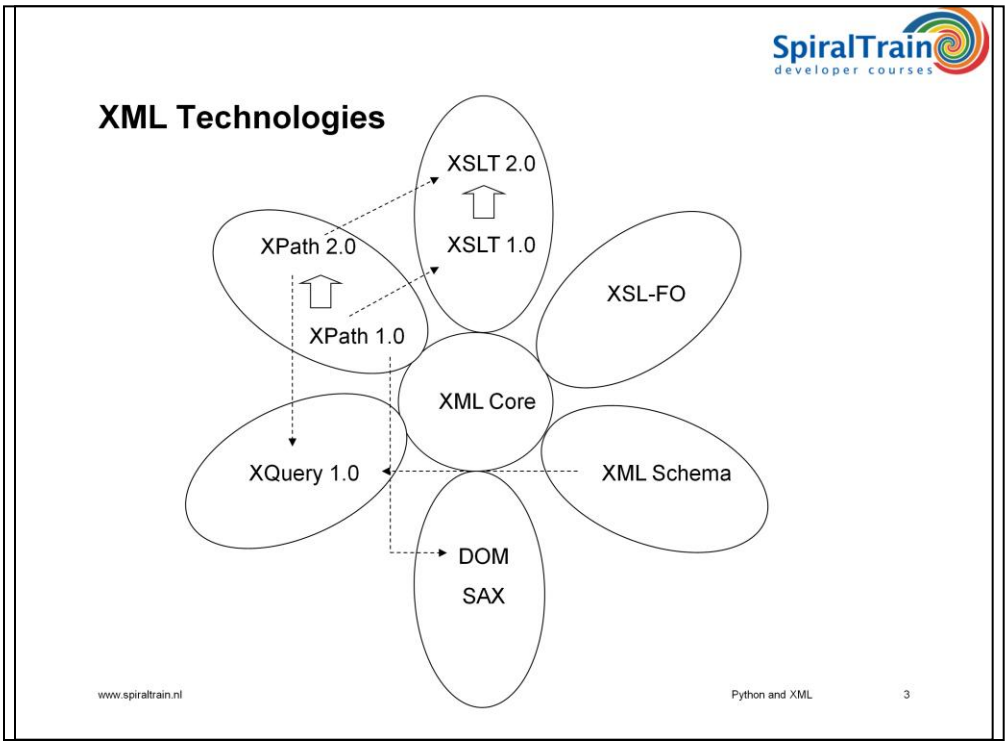
- XML Parsing
- XML Technologies
- XML Processing Options
- Python XML Libraries
- The XML DOM
- Building a DOM tree
- DOM Navigation
- DOM Manipulation
- Simple API for XML
- SAX Callbacks
- XML Parsing Models
- Pull versus Push Parsing
- XPath
- XPath Expressions

© copyright : spiraltrain@gmail.com



A DOM parser creates an in memory tree of an XML documents. After that you can use the DOM API to manipulate the object model in any way that makes sense. This mechanism is also known as the "random access" protocol, because you can visit any part of the data at any time. You can then modify the data, remove it, or insert new data.

A SAX parser fires events while reading an XML document. The basic technique is to register your handler with a SAX parser, after which the parser invokes your callback methods whenever it sees the event you register, for example a new XML tag



NOTE: Over 300 different XML related acronyms (and counting) :

The W3C XML Specification 1.0 relates only to syntax. It does not contain information about presentation, application or semantics of the XML. Other standards and specifications have been developed around the XML 1.0 specification.

It is important to understand that there are currently hundreds of XML specifications and initiatives. Some of these specifications and initiatives are currently in direct conflict with each other.

XML core :

Basic XML syntax and specification

XSLT (extensible stylesheet language) :

Used to transform xml data into other formats or other xml files

XPath (extensible path language) :

Used to navigate and extract information

XQuery :

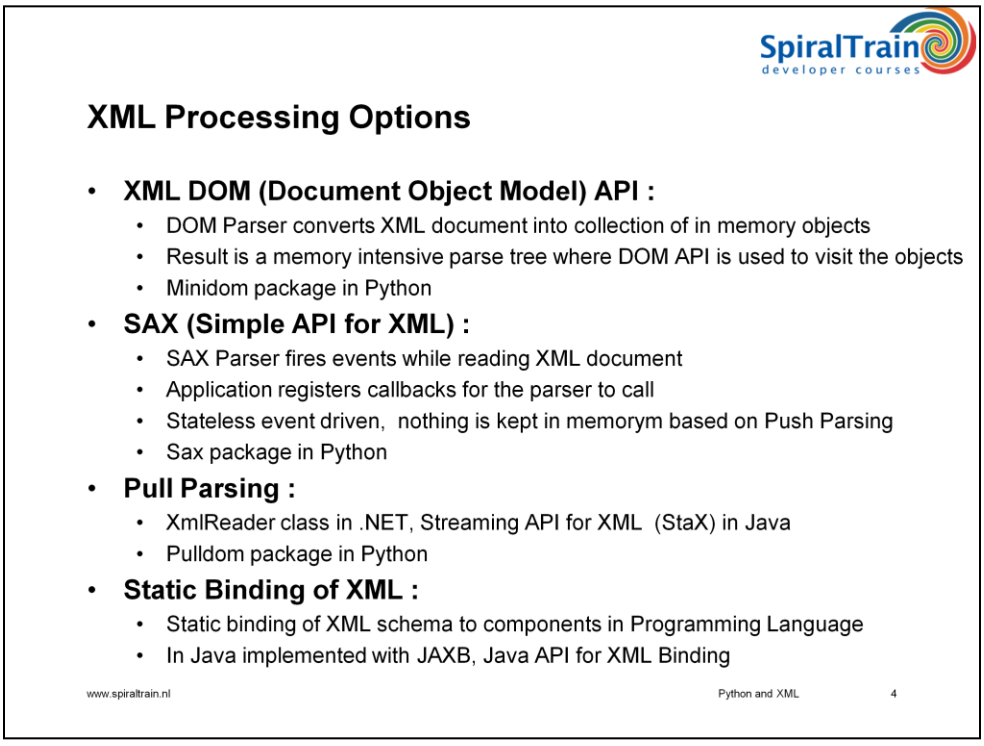
Used to query xml data like SQL for relational data

XML Schema :

Used to validate XML data

DOM and SAX :

Programmatic interface for accessing XML data



SAX provides an event-based paradigm for traversing XML. With SAX, the given XML is traversed in its entirety, from top to bottom, sequentially. As the XML is processed, methods are called on your class providing information for each node, its attributes, and data, in the order that they appear in the XML.

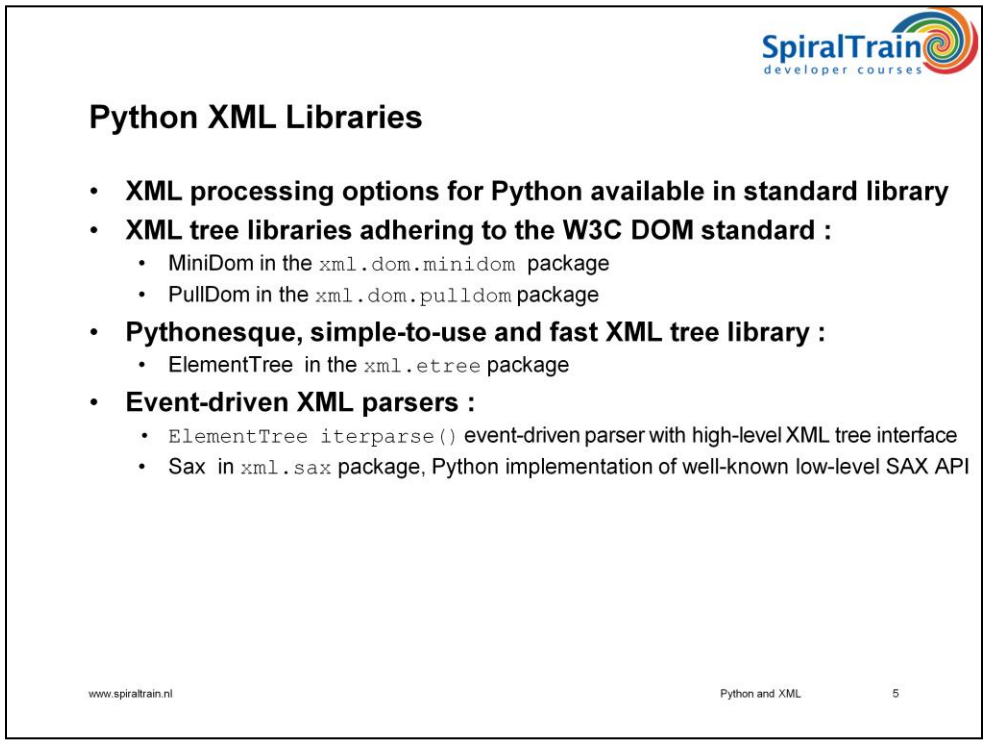
DOM parsing provides an alternative to SAX, letting your code locate a specific node in an XML document while ignoring the rest. The DOM also supports creating XML documents within your code, which you can serialize as a file on disk.

Although these two features make DOM parsing seem much more appealing than SAX processing, they come with a price. DOM parsing typically uses more memory and time to process XML, as it must create an object hierarchy in memory for the entire XML document before your application can use it. If the XML you are parsing is particularly large, this may not be an option.

In most cases, however, DOM parsing is the most efficient and useful way of working with XML in an application.

Both of these models give little or no control to the user in this parsing process. Once started, whether tree-based or event-based, both parsing approaches consume the whole data stream at once. JSR 173 defines a pull streaming model, *StAX* (short for "Streaming API for XML"), for processing XML documents. In this model, unlike in SAX, the client can start, proceed, pause, and resume the parsing process. The client has complete control.

The Java Architecture for XML Binding provides a powerful and practical way of working with XML content from within Java applications. The JAXB binding compiler (or xjc) lets you generate Java classes from a given XML schema. The JAXB binding compiler transforms an XML schema into a collection of Java classes that match the structure described in the XML schema. These classes are annotated with special JAXB annotations, which provide the runtime framework with the mappings it needs to process the corresponding XML documents.



The DOM and SAX packages have the advantage of being compatible with standard or de facto standard APIs, so users who are already familiar with these APIs can use them without learning too many new things. Everyone else should start with the faster and more pythonic ElementTree library, which is very well integrated into the Python language, and therefore very easy to learn and use.

External packages

A long list of special purpose and general purpose Python XML packages is available from PyPI. The following is a choice of major tools that support a broader set of XML features.

Pythonic tools

lxml - a pythonic, ElementTree-compatible binding for the libxml2 and libxslt libraries that comes with all sorts of powerful XML (and HTML) tools, well integrated into an easy-to-use Python API

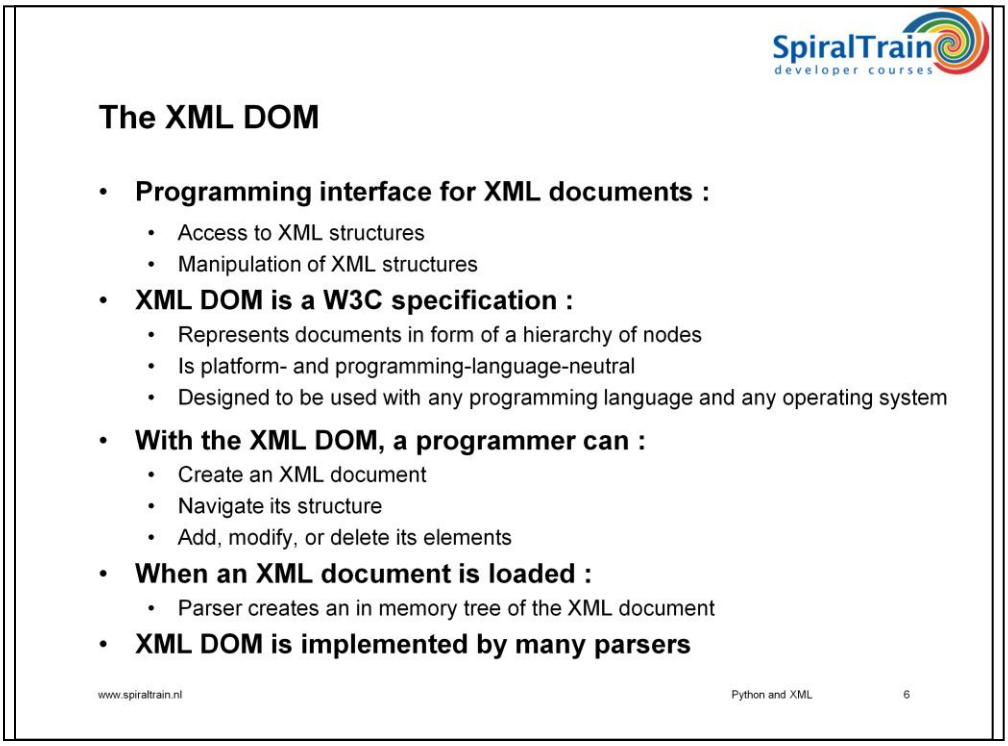
lxml.objectify - a Python object API for XML based on lxml

PyXB - generates Python classes/modules that correspond to data structures/namespaces defined by XMLSchema, with validation

PyXSD - an XML Schema mapping too (somewhat dated, last released in 2006)

generateDS - generates Python data structures (for example, class definitions) from an XML Schema document

Amara 2.x - Amara provides tools you can trust to conform with XML standards without losing the familiar Python feel

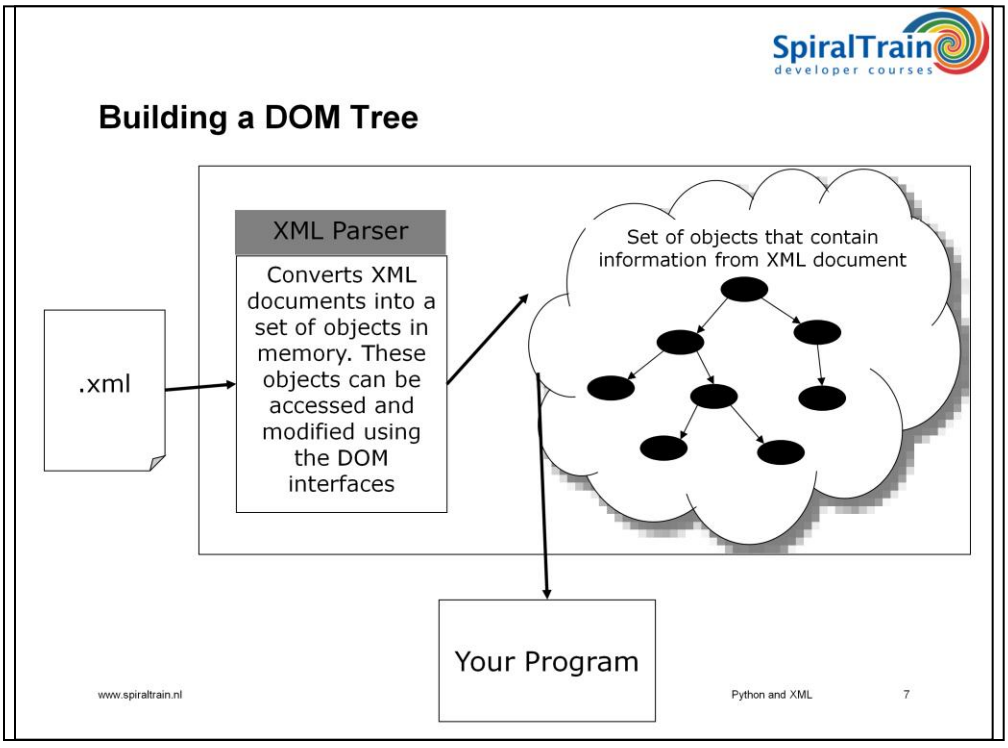


Document Object Model (DOM) is a platform- and language-independent standard object model for representing HTML or XML and related formats. Strictly speaking, one should refer to "the DOM", but in practice, the definite article is usually dropped.

Because DOM supports navigation in any direction (e.g., parent and previous sibling) and allows for arbitrary modifications, an implementation must at least buffer the document that has been read so far (or some parsed form of it). Hence DOM is likely to be best suited for applications where the document must be accessed repeatedly or out of sequence order. If the application is strictly sequential and one-pass, the SAX model is likely to be faster and use less memory.

The World Wide Web Consortium (W3C) developed the W3C Document Object Model[1] in response to the development of various proprietary models for HTML, particularly those used in web browsers. The existing vendor-specific interfaces were dubbed intermediate DOMs.

W3C began development of the DOM in the mid-1990s. Although the W3C never produced a specification for DOM 0, it was nonetheless a partially documented model and was included in the specification of HTML 4. By October 1998, the first specification of DOM (DOM 1) was released. DOM 2 was issued in November 2000, with specifics on the style sheet object model and style information manipulation. DOM 3 was released in April 2004 and is the current release of the DOM specification.



The Document Object Model (DOM) is a standard application-programming interface (API) designed by the W3C explicitly for accessing the content of HTML and XML documents.

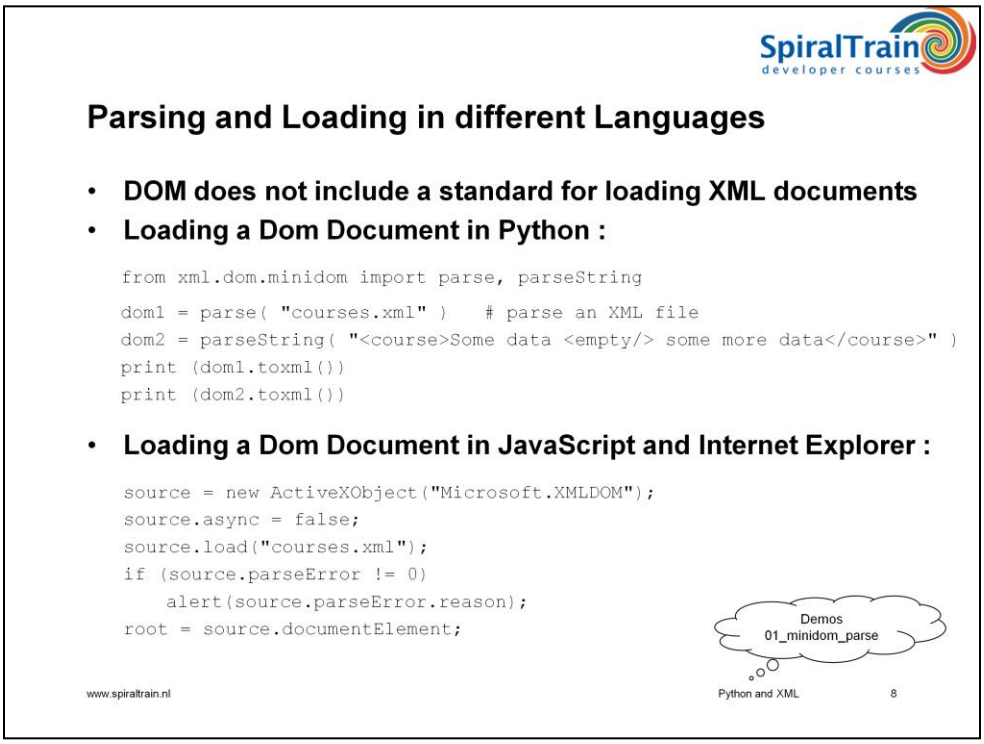
DOM defines the logical structure of documents using a system of Node objects arranged into a tree-like organization. DOM also specifies the way a document is accessed and manipulated from a variety of programming languages.

In the DOM Recommendation, the model is based on the content of a document. However, that document could contain XML that is being widely used for interchanging data between different systems. In the context of a web browser, that document would contain HTML.

Given access to a DOM representation of a document, a software developer can construct new documents or modify existing ones. Typically, within a web browser we are concerned more with modification. In general, the creation of document content supported by web browsers is weak.

The DOM standards developed by the W3C are language independent. Techniques that work in JavaScript may well work in Java (assuming a similar and compliant implementation exists for both languages). The W3C standards include bindings for JavaScript and Java in the appendices, while other groups are developing language bindings for different languages.

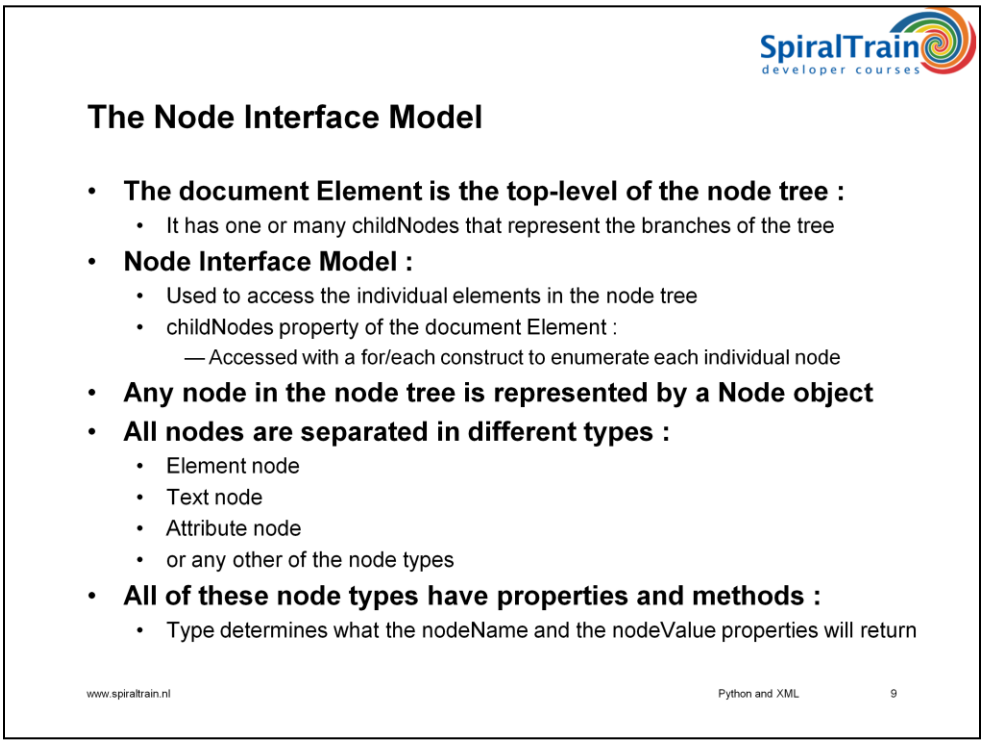
All of this suggests that without the DOM standards, we would be nowhere at all. In the early days, the browser implementations led the way, but the W3C is now much further ahead. The situation now, is that each browser supports its own particular DOM representation. They are converging with one another by adding capabilities described by the W3C, but you still have to write DOM access code very carefully to work around the shortcomings in the various implementations. Occasionally, a DOM specified feature helps you out, but generally the traversal support is implemented so differently in each browser, that you may still have to write browser specific code.



Sun has strategy of using factory classes to initiate the parser a bit different. Here are the necessary lines.

```
DocumentBuilderFactory dbf =
DocumentBuilderFactory.newInstance();
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(args[0]);
```

The first line creates a new DocumentBuilderFactory instance. By default, the DocumentBuilderFactory uses the built-in XML parser that comes with JAXP, but you can change the parser by setting the system property javax.xml.parsers.DocumentBuilderFactory. You can pre-configure the parser using factory class methods like setValidating() and setNamespaceAware(). After you have the factory, you create a DocumentBuilder and invoke its parse() method to parse a document.



The primary purpose of the Node interface is to define the base functionality for all node types. It defines the set of attributes, methods, and constants that must be available on any node within the DOM hierarchy. This makes it possible to traverse a DOM hierarchy in a uniform fashion strictly using the Node interface without having to downcast to more specific interface types. The more specific interfaces are available when necessary to access features that only make sense for a given node type.

One of the tradeoffs of a fairly general base interface like Node is that some of its operations do not apply to all of the node types. The DOM working group made a conscious decision to not factor the Node interface to the smallest common subset. Rather, the DOM working group tried to balance the benefits of type safety with the convenience and uniformity of a single model-wide interface. This compromising of type-safety means that some errors may not be detected until runtime rather than at compile-time.

For generality, the DOM does not rely on the use of runtime type identification (RTTI) in the target programming language. Rather, the nodeType attribute is used to test a node for compatibility with a derived interface (for example, Element, Document). The Node interface also defines a group of symbolic constants that correspond to the different node types

SpiralTrain
developer courses

Node Types

nodeType	nodeTypeString	nodeName	nodeValue
1	element	tagName	null
2	attribute	name	value
3	text	#text	content of node
4	cdatasection	#cdatasection	content of node
5	entityreference	entity reference name	null
6	entity	entity name	null
7	processinginstruction	target	content of node
8	comment	#comment	comment text
9	document	#document	null
10	documenttype	doctype name	null
11	documentfragment	#document fragment	null
12	notation	notation name	null

The document object is the root element in the node tree. All nodes in the node tree are childNodes of the document element. The document element is required in all XML documents.

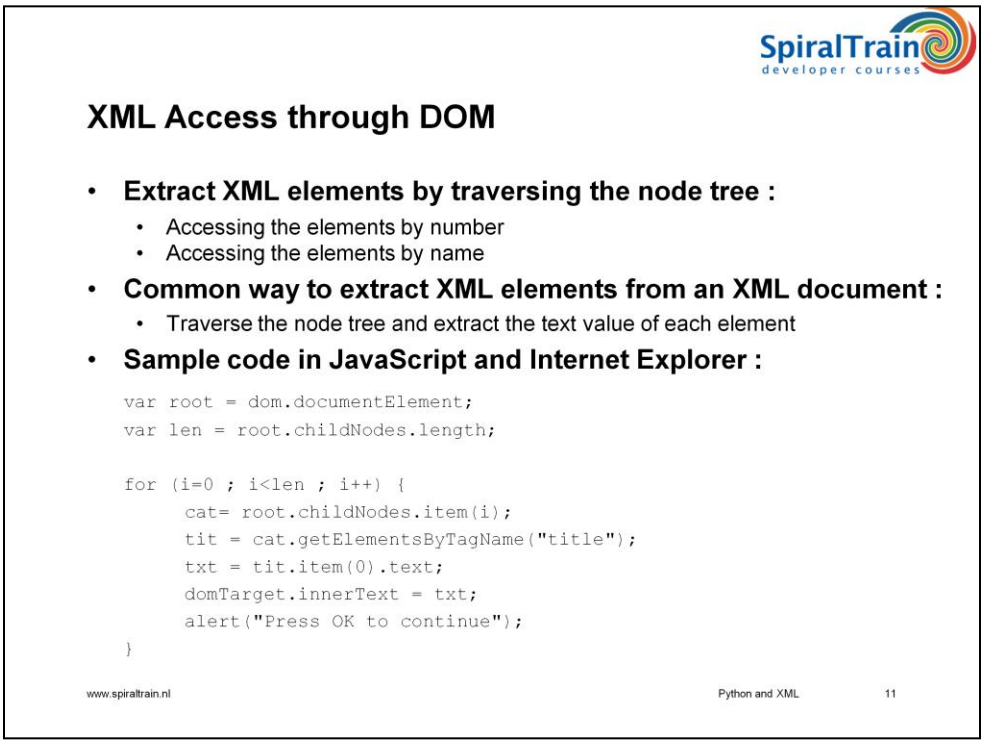
The element object represents the element nodes in the document. If the element node contains text, this text is represented in a text node.

The attr object returns an attribute of an element object as an attribute node. The attr object has the same properties and methods as nodes in general.

The text object represents the text inside an element as a node.

The CDATASection object represents the CDATASection nodes in a document. The CDATASection node is used to escape parts of text which normally would be recognized as markup.

The comment object represents the comment nodes in a document. The comment nodes have no nodeName, but their nodeValue is the comment text.

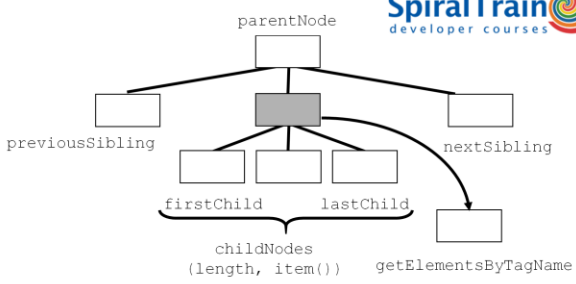


When the XML document is loaded, you can move on to processing the XML data. However, there is browser inconsistency to deal with. The inconsistency has to do with how the two major browsers load XML documents. You will typically create a JavaScript function that processes XML data, and you'll likely want this function to get called right after loading the documents.

Following is the code to handle calling a function named printRootNode() upon loading the XML document courses.xml:

```
if (window.ActiveXObject) {
  xmlDoc = new ActiveXObject("Microsoft.XMLDOM");
  xmlDoc.load("courses.xml");
  printRootNode();
}
else {
  xmlDoc = document.implementation.createDocument("", "", null);
  xmlDoc.load("courses.xml");
  xmlDoc.onload = printRootNode;
}
```

DOM Navigation



Property	Description
childNodes	A collection of node objects that are immediate children of the owning node
firstChild	The first item in the childNodes collection
lastChild	The last item in the childNodes collection
parentNode	A reference to the node whose childNodes collection this node belongs to
previousSibling	Node immediately before current node in childNodes collection belonging to their joint parent node
nextSibling	Node immediately after current node in childNodes collection belonging to their joint parent node
getElementsByTagName (tagName)	Returns a collection of objects having been instantiated from the specified tag name

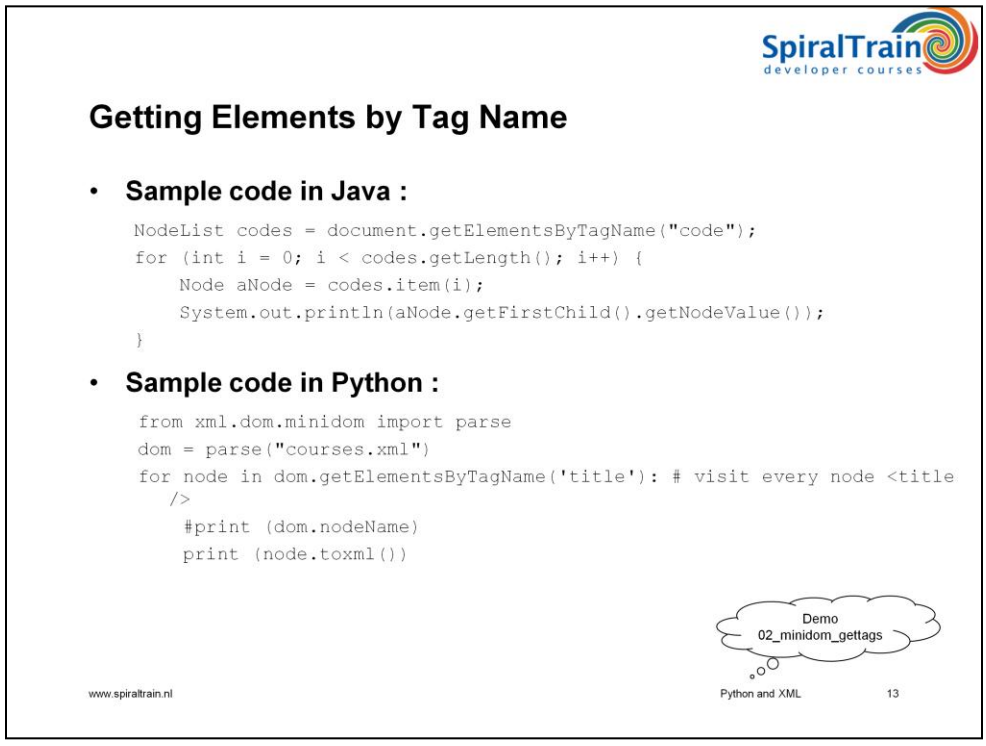

```
source = new ActiveXObject("Microsoft.XMLDOM");
source.async = false;
source.load("catalog.xml");

if (source.parseError != 0) alert(source.parseError.reason);

root = source.documentElement;
alert("Look at the status bar for information ")
status = "Looking at the root element";
alert(root.nodeName);
alert(root.nodeType);

status = "Finding the authors";
book2 = root.childNodes.item(1);
b=root.childNodes.length;
alert(b);

for (i=0;i<4;i++) {
   x= root.childNodes.item(i);
   t = x.getElementsByTagName("title");
   alert(t.item(0).text);
}
```



The getElementsByTagName method returns a NodeList collection of those elements with the tag name specified as the argument. If you use the value "*" as the argument, then a list of all elements is returned.

Sample Code JavaScript :

```
var xml_doc = new ActiveXObject("Microsoft.XMLDOM");
xml_doc.async = false;
xml_doc.load("passeriformes.xml");

var i, n_elems, elems = xml_doc.getElementsByTagName("species");
n_elems = elems.length;
for (i = 0; i < n_elems; i++)
   document.write(elems[i].firstChild.nodeValue + "<br>");
```

SpiralTrain developer courses

DOM Manipulation

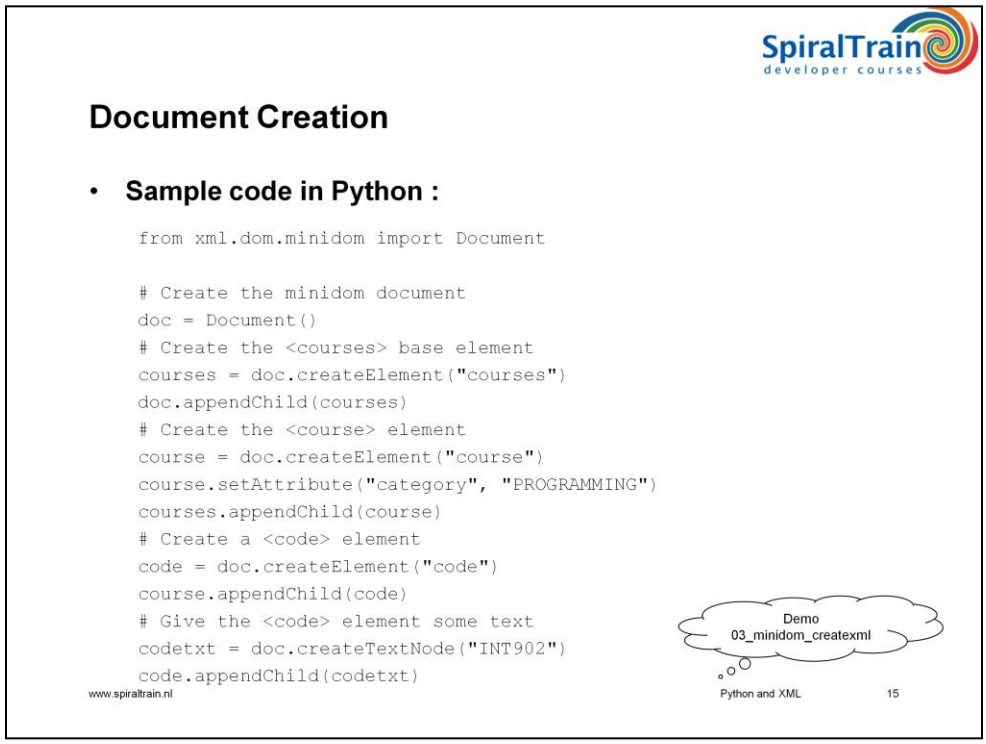
Method	Description
`createElement(tagName)`	Creates new instance of Element node based on tag name
`createTextNode(aText)`	Creates a new instance of a Text node containing the specified text as its data
`appendChild(aNode)`	Append a node to the receiving Node object's childNodes collection
`insertBefore (newNode, oldNode)`	Insert a new node into the childNodes collection of the receiving node immediately before the indicated oldNode
`removeChild(aNode)`	Remove a node from the receiving node's childNodes collection
`replaceChild (newNode, oldNode)`	Replace an old node with a new one in the receiving nodes' childNodes collection


```
newBook = source.createElement("book");
newTitle = source.createElement("title");
newText = source.createTextNode("XML Design and Implementation");
newTitle.appendChild(newText);
newBook.appendChild(newTitle);
root.appendChild(newBook);
alert(root.xml);

status = "setting a nodeValue";
newText.nodeValue = "XML Design and Implementation, Second Edition";
alert(newText.data);
alert(root.xml);

alert(book1.parentNode.nodeName);
alert(book1.nextSibling.getElementsByTagName("title").item(0).text);

status = "Getting all the attributes of a node";
alert(price.attributes.item(0).value);
```

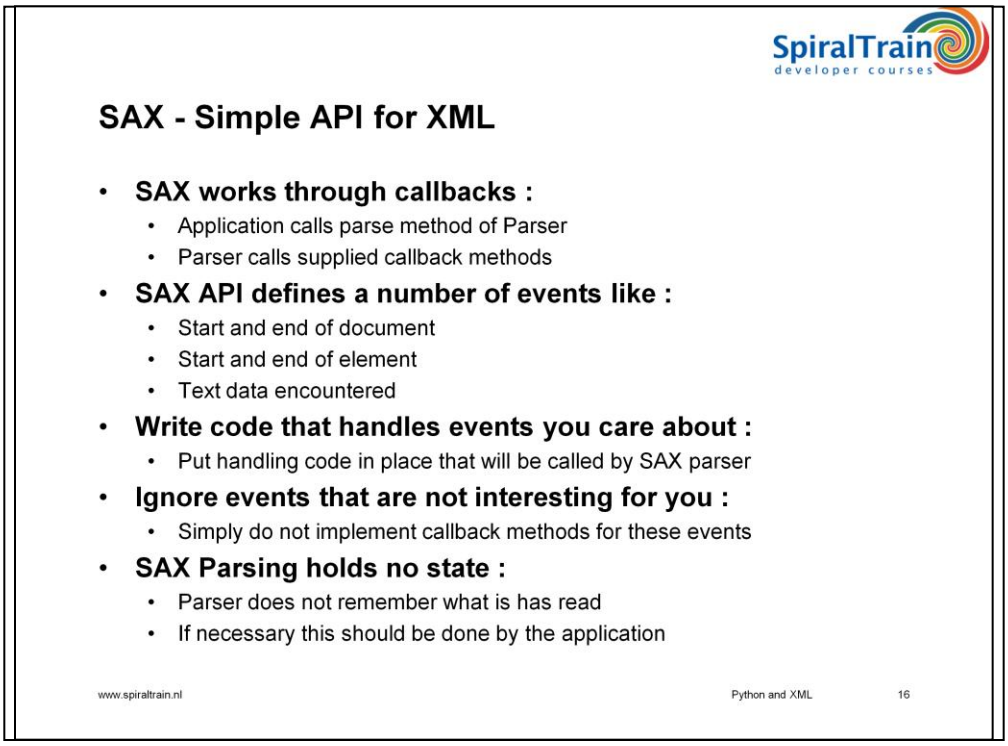


In .NET there are two ways to create an XML document. One way is to create an XmlDocument with no parameters. The other way is to create an XmlDocument and pass it an XmlNameTable as a parameter. The following example shows how to create a new, empty XmlDocument using no parameters.

```
XmlDocument doc = new XmlDocument();
```

Once a document is created, you can load it with data from a string, stream, URL, text reader, or an XmlReader derived class using the Load method. There is also another load method, the LoadXML method, which reads XML from a string.

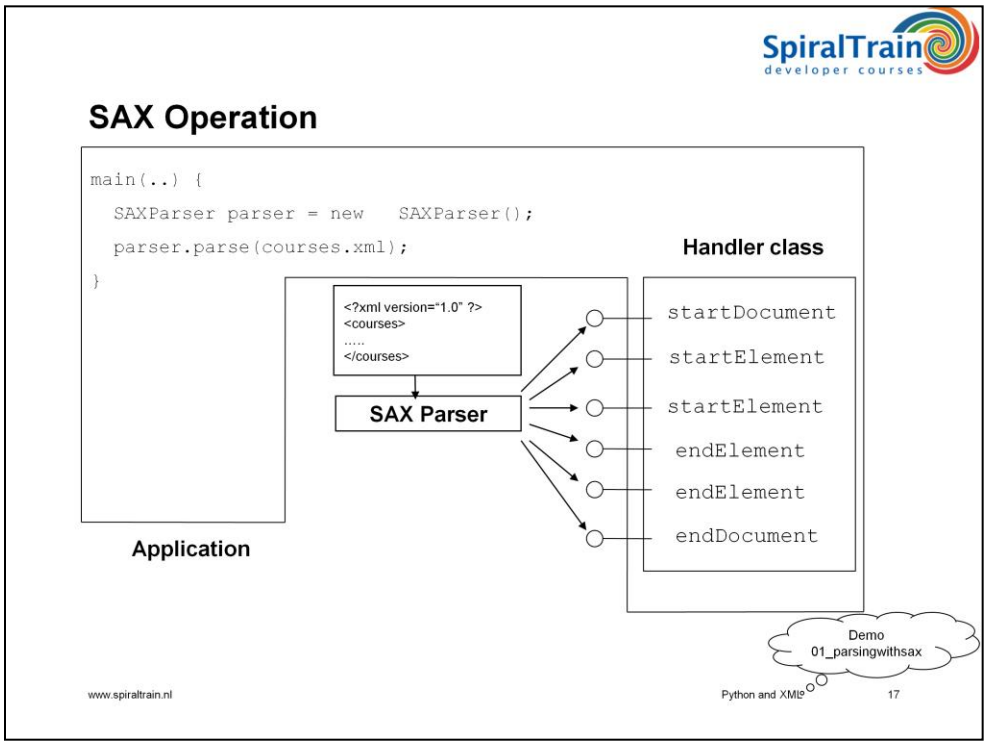
There is a class called the XmlNameTable. This class is a table of atomized string objects. This table provides an efficient means for the XML parser to use the same string object for all repeated element and attribute names in an XML document. An XmlNameTable is automatically created when a document is created as shown above and is loaded with attribute and element names when the document is loaded. If you already have a document with a name table, and those names would be useful in another document, you can create a new document using the Load method that takes an XmlNameTable as a parameter. When the document is created with this method, it uses the existing XmlNameTable with all the attributes and elements already loaded into it from the other document. It can be used for efficiently comparing element and attribute names.



Simple API for XML (SAX), an event-driven XML API, defines a stream of events specifying XML structure as handed from the parser to the specialized handler code. You register handler code for specific events that are triggered by different parts of XML markup (such as start and end tags, text, and entities). The parser then sends a stream of these events based on the input XML, which the handler code processes in turn.

SAX was essentially created on a marathon thread starting in late 1997 on the XML-DEV mailing list, which has long been the prime habitat for XML experts. David Megginson led the discussion, and the result was one of the most successful XML initiatives, with no large company or standards-body sponsorship. Before SAX, each parser had its own peculiar API for communicating XML structure to handler code, and SAX provided important unification. In general, parsers provide SAX drivers that translate low-level parser events into SAX standard events, allowing for portable code. SAX was developed with the Java language in mind, but it has become popular across numerous languages and environments, although sometimes its Java-centricity complicates porting.

In mainstream languages, event-based interfaces are usually implemented using callback functions, a style familiar in graphical user interface (GUI) programming and the like. In object- oriented languages, callbacks are usually registered methods for an object, using polymorphism to match the method name to the handler code, and using encapsulation to manage state in the handler between callbacks. This overall model of event-based programming is known as a push model and has a reputation for being difficult for many programmers to master. Most models that are considered easier to program, however, require random access to the document, and thus can lead to inefficiencies, so SAX has the reputation for being the most efficient standard way to process XML, if far from the easiest.

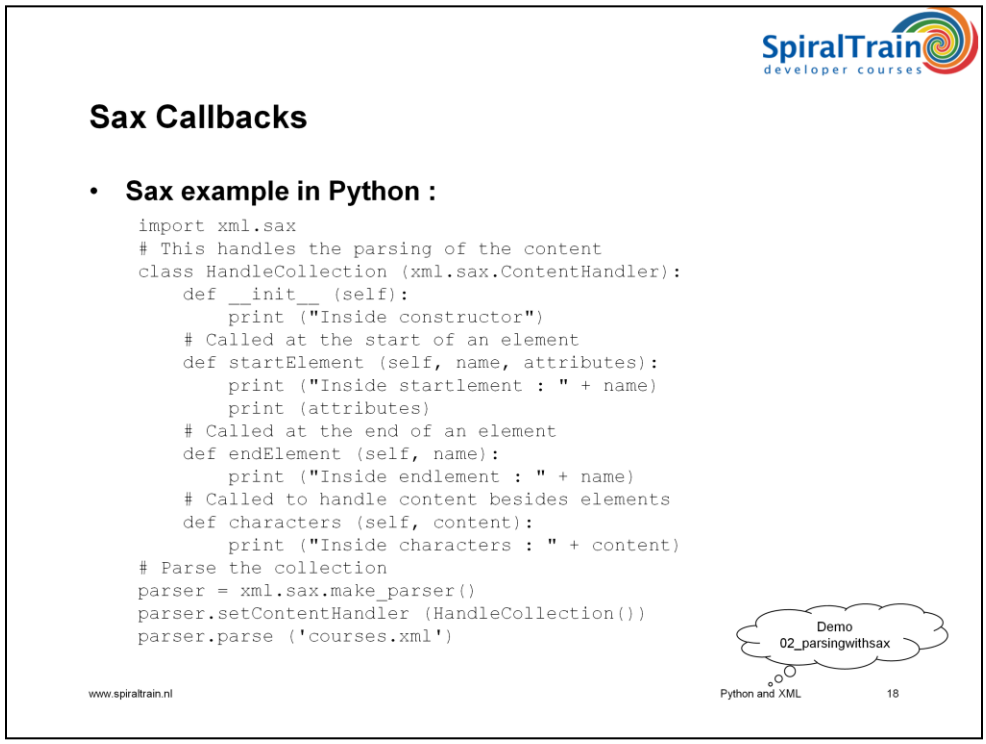


The SAX parser calls startElement() whenever it encounters an open tag, and endElement() whenever it encounters a close tag. These methods often contain the code that does the majority of the work while parsing an XML file. startElement()'s first argument is a string, which is the tag name of the element encountered. The second argument is an object of type AttributeList that provides sequential or random access to element attributes by name.

The characters() method indicates character content in the XML document characters that don't appear inside an XML tag, in other words. This method's signature is a bit odd. The first argument is an array of bytes, the second is an index into that array indicating the first character of the range to be processed, and the third argument is the length of the character range.

It might seem that an easier API would have simply passed a String object containing the data, but characters() was defined in this way for efficiency reasons. The parser has no way of knowing whether or not you're going to use the characters, so as the parser parses its input buffer, it passes a reference to the buffer and the indices of the string it is viewing, trusting that you will construct your own String if you want one. It's a bit more work, but it lets you decide whether or not to incur the overhead of String construction for content pieces in an XML file. The characters() method handles both regular text content and content inside CDATA sections, which are used to prevent blocks of literal text from being parsed by an XML parser.

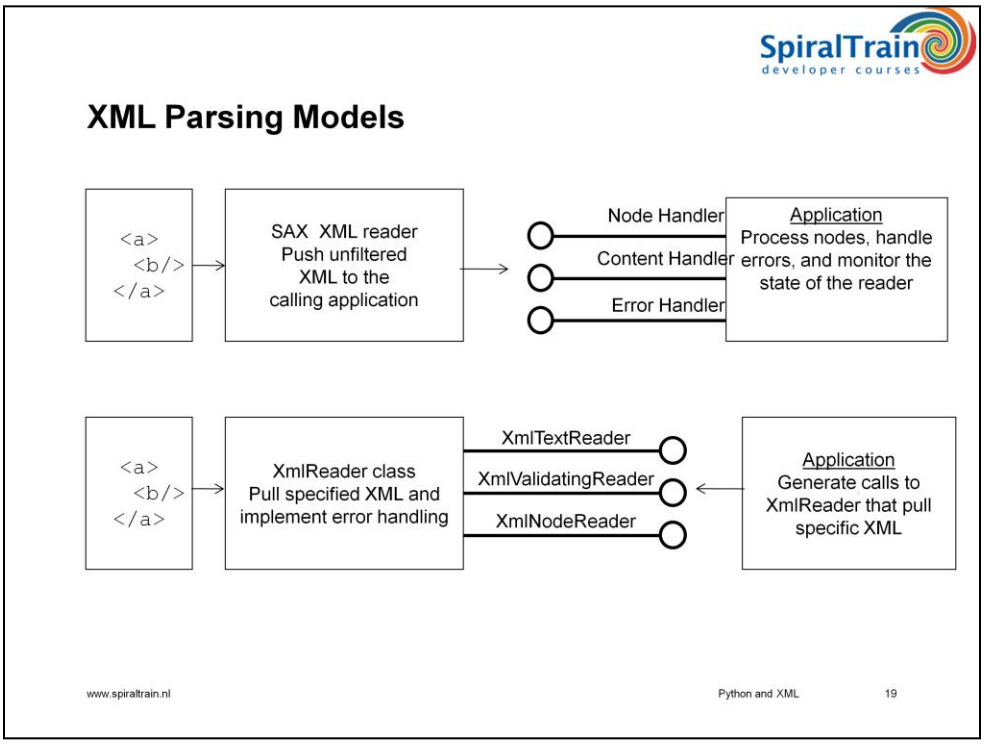
There are three other methods : ignorableWhitespace(), processingInstruction(), and setDocumentLocator(). ignorableWhitespace() reports occurrences of white space, and is usually unused in nonvalidating SAX parsers. processingInstruction() handles most things within <? and ?> delimiters; and setDocumentLocator() is optionally implemented by SAX parsers to give you access to the locations of SAX events in the original input stream.



SAX uses the Observer design pattern to tell client applications what's in a document. Java developers are most familiar with this pattern from the event architecture of the AWT and Swing.

In that context, the client programmer implements an interface such as MouseListener that receives events through well-known methods. Then the programmer registers the MouseListener object with the component such as a Button using the setMouseListener() method. When the end user moves or clicks the mouse in the button's area, the button invokes a method in the registered MouseListener object. In this example, the Button class plays the role of the Subject, the MouseListener interface plays the role of the Observer, and the client-defined implementation of that interface plays the role of the ConcreteObserver.

SAX works in a very similar way. However in SAX, XMLReader plays the role of Subject and the org.xml.sax.ContentHandler interface plays the role of Observer. The biggest difference between the AWT and SAX is that SAX does not allow more than one listener to be registered with each XMLReader. Otherwise, the pattern is exactly the same.

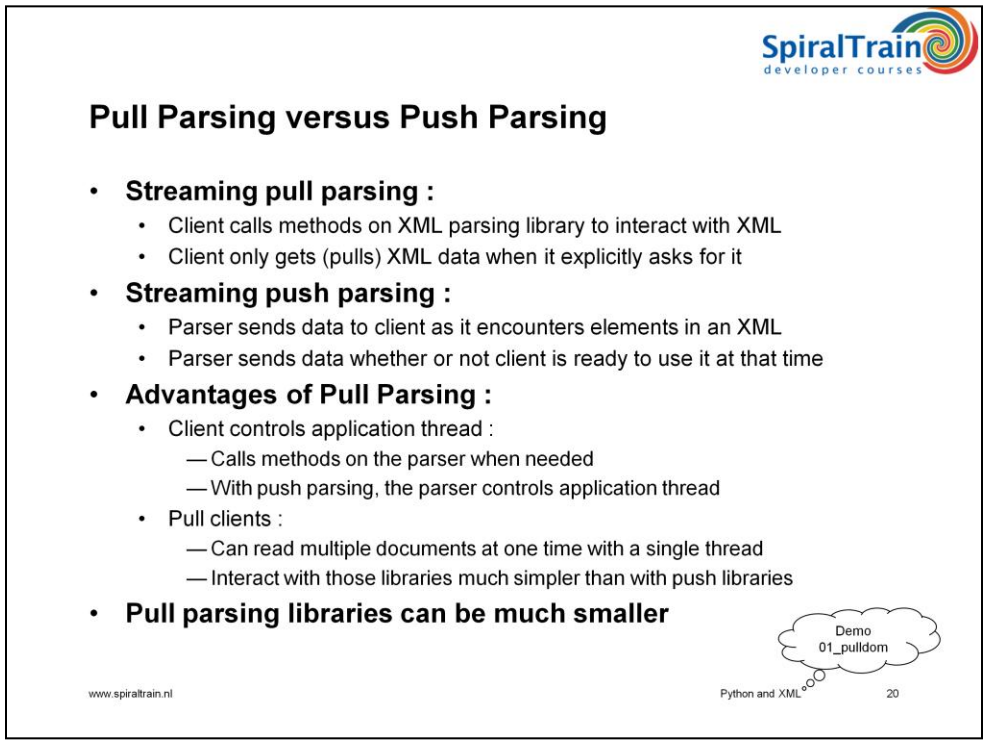


Parsing with SAX

One of the disadvantages of SAX can be that you will have to implement all the event handlers to handle each and every incoming event. The application code must be maintained in this state of events. The SAX parser is incapable of processing the events when it comes to the DOM's element supports, and you also have to keep track of the parsers position in the document hierarchy. The application logic gets tougher as the document gets complicated and bigger. It may not be required that the entire document be loaded but a SAX parser still requires to parse the whole document, similar to the DOM.

Parsing with STax

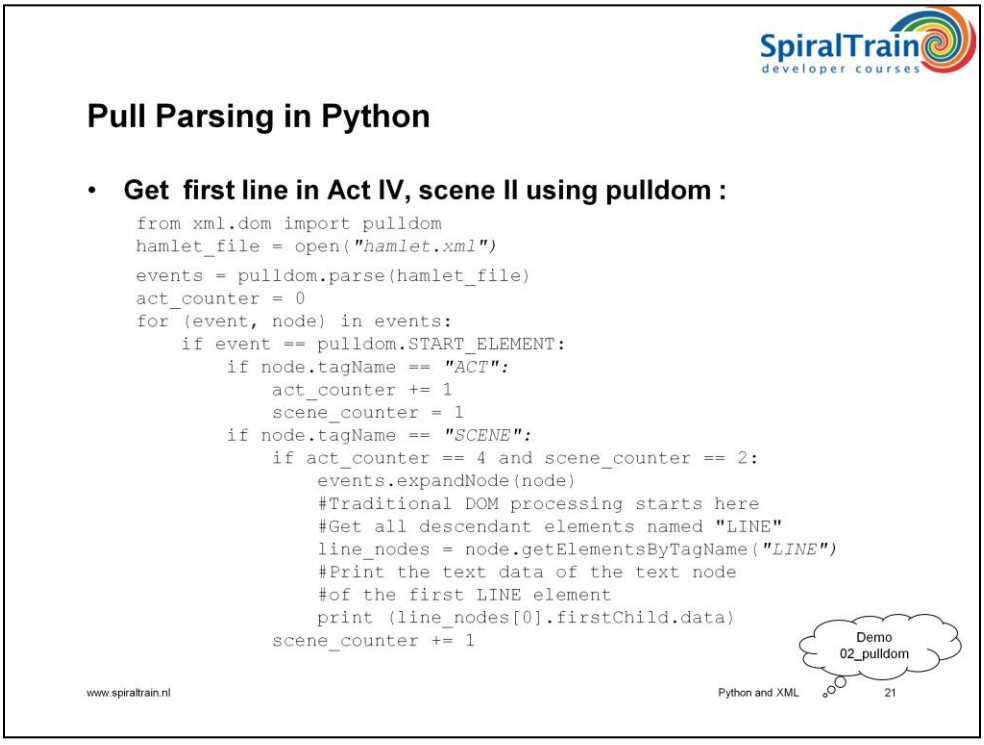
Stax is a brand new parsing technique which is very similar to SAX and also an improvisation to it. The STAX uses a model that is event-driven. The only difference between sax and STAAX here is that the sax uses a push model and the STAX uses a pull model for event processing. And also another notable feature is instead of using call back options the STAX parser returns events which are requested by the applications in use.



With so many XML technologies, deciding what to use and when to use it can sometimes be bewildering. Many chose to build on top of existing DOM or SAX implementations rather than StAX (the Streaming API for XML).

StAX is a parser-independent, streaming pull-based Java API for reading and writing XML data. It is a memory-efficient, simple, and convenient way to process XML while retaining control over the parsing and writing process. Most parsers fall into two broad categories: tree based (e.g., DOM) or event based (e.g., SAX). Although StAX is more closely aligned with the latter, it bridges the gap between the two. In SAX, data is pushed via events to application code handlers.

In StAX, the application "pulls" the data from the XML data stream at its convenience. Application code can filter, skip tags, or stop parsing at any time. The application--not the parser--is in control, which enables a more intuitive way to process data.



XSLT Support

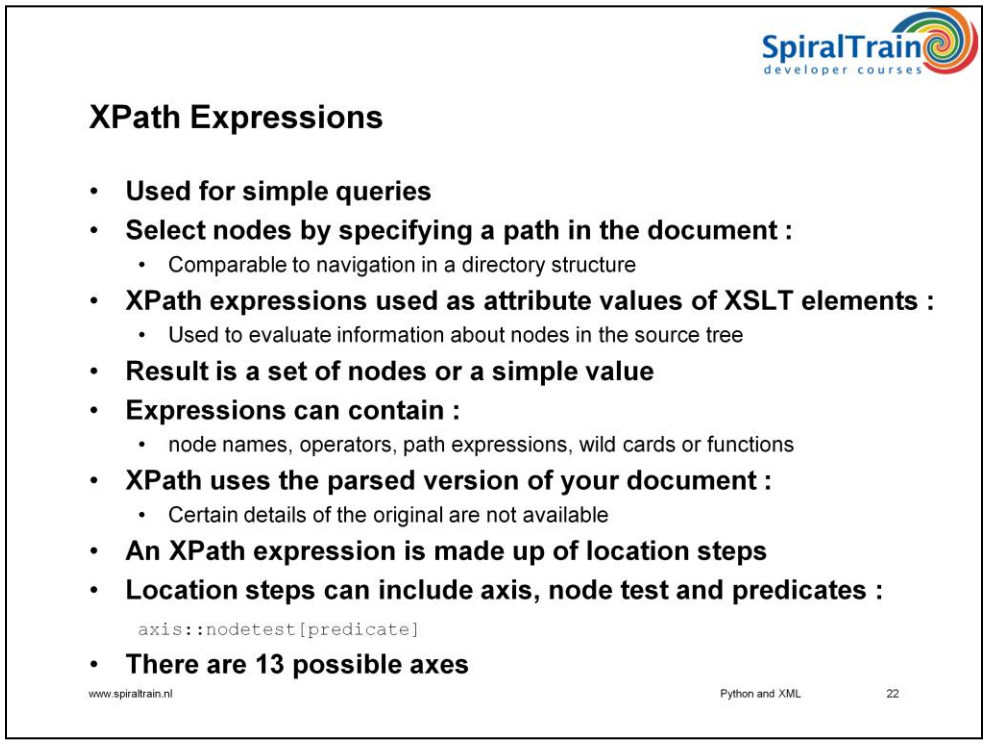
If not mentioned otherwise, this means XSLT 1.0, not XSLT 2.0.

lxml has excellent (and easy-to-use) XSLT support that is based on libxslt. It also supports calling into Python code from XSL transformations through both XPath and XSLT extensions.

XSLTools - XSL transformations on top of libxslt and libxml2dom, with added Web development support

Some tools linked from the XQuery homepage provide Python bindings for their XSLT2 and XPath2 implementations

Amara 2.x - Amara exposes an API to fully-compliant XSLT (including EXSLT)

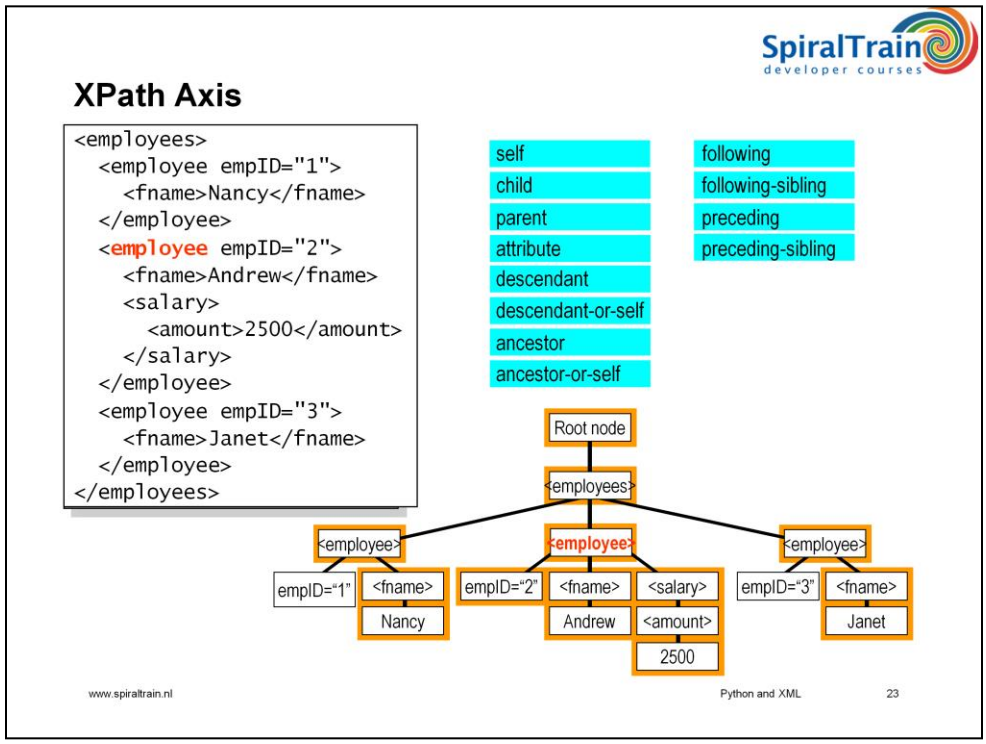


XPath is a language for addressing parts of an XML document. XPath 2.0 is a much more powerful language that operates on a much larger domain of data types. A better way of describing XPath 2.0 is as an expression language for processing sequences, with built-in support for querying XML documents.

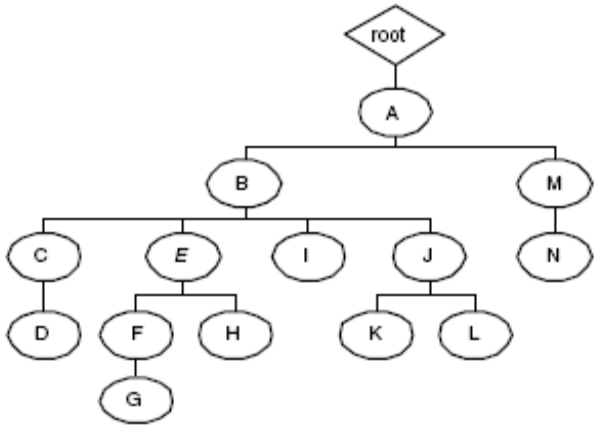
If you recall, XPath 1.0 supported only four expression types : node-set, boolean number (floating-point), string. This had the value of being simple but the disadvantage of being limited when it came to processing typed values, such as dates. XPath 2.0, on the other hand, introduces support for the XML Schema primitive types, which immediately gives the user access to 19 simple types, including dates, years, months, URIs, etc. In addition, a number of functions and operators are provided for processing and constructing these different data types.

XPath 2.0 expressions can return simple-typed values, nodes, or sequences of nodes or simple-typed values. Actually, every expression returns a sequence, as we will see.

Nodes in XPath 2.0 have the same basic definition as in XPath 1.0, except that certain kinds (elements and attributes) can now be associated with XML Schema types and processed as such. As in XPath 1.0, there are seven node types: document nodes, elements, attributes, namespace nodes, processing instructions, comments, and text nodes. The one difference in terminology here is that "root nodes" are now called, perhaps more appropriately, "document nodes".



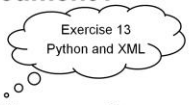
The figure illustrates the group of nodes identified by each axis, assuming E is the context node.



Axis	Nodes (relative to E)
self	E
parent	B
child	F,H
descendant	F,G,H
descendant-or-self	E,F,G,H
ancestor	B,A,root
ancestor-or-self	E,B,A,root
preceding	D,C
preceding-sibling	C
following	I,J,K,L,M,N
following-sibling	I,J

Summary : Python and XML

- **Python supports different XML processing options :**
  - Reading XML document in memory with DOM
  - Pull parsing and Push parsing
- **XML DOM parsing supported with Minidom package :**
  - Whole XML document is parsed and loaded into memory
- **SAX parsing supported with Sax package in Python :**
  - Parser fires events which are caught by application code
- **XML Pull Parsing supported with Pulldom package :**
  - Application is in control to retrieve XML parts from XML document
- **Pythonesque, simple-to-use and fast XML tree library :**
  - ElementTree in the `xml.etree` package
- **XPath selects nodes by specifying a path in the document :**
  - Comparable to navigation in a directory structure



© copyright : spiraltrain@gmail.com